

Subject: CS336 Assignment 1, ASCII / Unicode / UTF-8 Encoding

Date: from 2026/7/20 to 2026/7/21

Contents

ASCII, Unicode and UTF-8	1
---------------------------------	----------

ASCII, Unicode and UTF-8

ASCII and Unicode

计算机底层只能处理 0 和 1. 为了显示字符, 必须先约定“哪个数字代表哪个字符”.

- **ASCII.** 使用一个字节表示 127 个字符. 例如, 字符 A 对应十进制数字 65. ASCII 不能覆盖中文; 若向仅支持 ASCII 的系统输入“牛”, 系统会因为找不到对应编号而产生乱码.
- **Unicode.** 为世界各地的字符分配唯一编号, 这个编号称为码点 (code point). 例如, A 的码点仍为 65; 汉字“牛”的码点是 29275, 十六进制写作 U+725B.

Unicode 解决了字符的“身份编号”问题, 但并没有规定这些编号应当怎样存进内存或写入文件. 完成这一步需要具体的字符编码方案.

UTF-8, UTF-16 and UTF-32

UTF 是 *Unicode Transformation Format* 的缩写. 它负责把抽象的 Unicode 码点转换为计算机能够存储和传输的字节流.

编码	存储规则	优点	缺点
UTF-32	固定 4 字节	定位、读取简单	英文字符也占 4 字节, 空间浪费严重
UTF-16	2 或 4 字节	Windows、Java 中较常见	仍是变长编码, 处理边界较复杂
UTF-8	1-4 字节	节省空间, 兼容 ASCII	定位第 N 个字符时通常需要向前遍历

对于 A, UTF-8 只需 1 byte; 对于“牛”, UTF-8 需要 3 bytes. 由于英文字符在 UTF-8 中仍使用与 ASCII 相同的单字节表示, UTF-8 已成为网页、代码和跨平台文本最常用的编码方式.

How does UTF-8 split a code point into bytes?

UTF-8 使用前缀识别法: 观察首字节开头的若干位, 就能判断当前字符占用几个字节; 后续字节均以 10 开头.

码点范围	字节数	Byte 1	Byte 2	Byte 3	Byte 4
0000-007F	1	0xxxxxxx	-	-	-
0080-07FF	2	110xxxxx	10xxxxxx	-	-
0800-FFFF	3	1110xxxx	10xxxxxx	10xxxxxx	-
010000-10FFFF	4	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

Example. “牛”的码点为 U+725B, 位于 0800-FFFF, 因此使用三字节模板:

模板	1110xxxx	10xxxxxx	10xxxxxx
编码	11100111	10001001	10011011

最终字节序列是 E7 89 9B. 使用 Python 验证: “牛”.encode(“utf-8”) = E7 89 9B.